TRACE

Step-level trust monitoring for autonomous AI agents

Lumina (RA014) | Final plan: the idea, where we are strong, how we fit the judging criteria, and the task split

Our pitch in two sentences. An LLM produces or judges content; TRACE monitors an agent's *execution*, finds the step where trust first breaks, and shows the evidence for an intervention. We test one clear hypothesis: **step-level monitoring catches failures that final-answer-only checking misses**, and we report honestly where it fails.

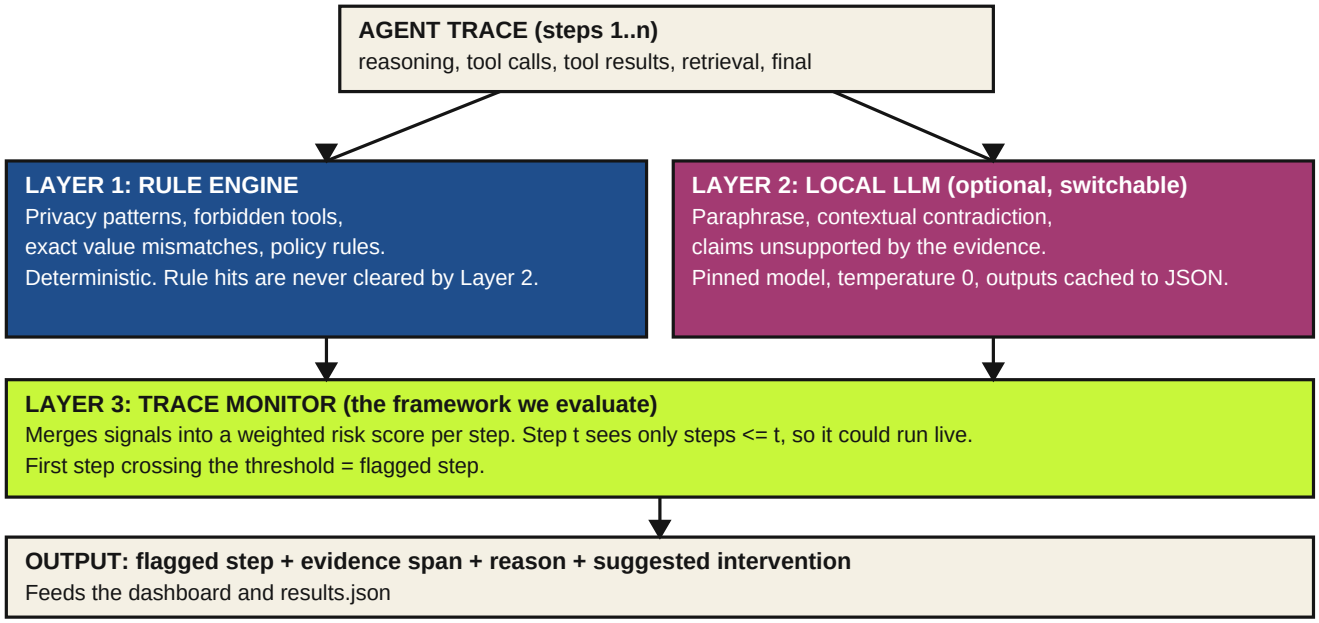
The problem

Autonomous agents reason, call tools and retrieve data with little human oversight, but we judge them almost entirely by their final answer. A private value can leak in a tool call, or a claim can contradict a tool result, while the final answer looks perfectly safe.

What we build (all offline, simulated traces, no paid model calls)

- **A trace format:** a JSON schema for one agent run, step by step.
- **A monitor** that scores every step on evidence, tool-result consistency, uncertainty and policy compliance, then flags the first unreliable step.
- **A hybrid design:** a deterministic rule engine plus an optional local LLM, merged by the TRACE monitor.
- **A dashboard/report** that shows each run's steps, the flagged step, and the reason, next to the final-answer-only baseline.
- **Intervention rules** (continue, verify, pause for human, abort) and an honest write-up of failures and limits.

Architecture



All layers emit the same signal record: `step_id, signal, score, evidence_span, source (rule | llm)`. Because Layer 3 only sees this record, the dashboard works the same with the LLM on or off, and the system still runs end to end on rules alone.

Where we stand: prior work and what is different

This is an active research area, so we do **not** claim to be first. From a literature search (October 2026), failure attribution in agent runs has several benchmarks and methods, and they are mostly post-mortem and LLM-judge based. *Verify these before presenting; we read abstracts, not full papers.*

Related work	What it does	How TRACE differs
AgentRx	Synthesizes constraints, checks them step by step, outputs an auditable violation log; an LLM judge then localizes the critical step	Closest to us. We add privacy-leak detection across channels, a causal per-step score, and an explicit final-answer-only comparison
Who&When and follow-ups (FALAT, TrajDebug, REFLECT)	Failure attribution: find the decisive error step in failed multi-agent runs, mostly with LLM judges	We score every step as it happens instead of diagnosing a failed run afterwards, and our core needs no LLM
Traverse, CodeTracer	Large benchmarks of real agent trajectories with human-labelled first-mistake steps	We use small simulated traces, so we claim a validated prototype, not benchmark-level evidence
Observability tools (LangSmith, Langfuse, etc.)	Log and visualize traces (from general knowledge, not checked in our search)	They record; TRACE scores trust per step and recommends interventions

Where we are strong

Strength	Why it matters to the panel
The blind-spot demo	A trace where the final answer is safe but a middle step leaked private data. Final-answer-only checking misses it; TRACE flags the exact step. This is our clearest, most visual win.
A testable hypothesis	We compare four systems on the same traces (below) and say in advance what result would change our conclusion.
Interpretable by construction	Rule hits point to a rule id and a matched span. LLM judgements must return a quoted evidence span, shown with a source badge in the dashboard.
Honest evaluation design	Held-out traces, single-signal ablations, a baseline that can win (T10), and required write-ups of one miss and one false alarm.
Reproducible	One command, fixed seed, pinned model, cached LLM outputs, a fresh-clone test. Anyone gets the same numbers without running the model.
Runs without the LLM	If the LLM layer fails, the rules-only system is still complete. The LLM is an experiment on top, not a dependency.

Where we are weak (and will say so)

- **Small, hand-written data.** About 13 to 20 simulated traces cannot support statistical or real-world claims.
- **Rules are brittle on paraphrase;** a local LLM can miss failures or invent them, and its outputs can vary.
- **Uncertainty is a documented proxy,** not the model's true uncertainty. Hidden-state probes from our abstract are out of scope without model access.
- **The combiner is a weighted risk score, not a calibrated one.** We use the word calibrated only if we run a calibration procedure.
- **Novelty is moderate.** The contribution is the combination, the leak-focused step-level view, and the rigorous comparison.

Scope: abstract versus prototype

Our abstract describes the research vision. The hackathon prototype implements a defined subset. We state this up front so nobody discovers it for us.

Abstract promise	What the prototype demonstrates
Multi-channel leak detection	Pattern scan across simulated reasoning, tool outputs and retrieval results
Uncertainty on intermediate reasoning	A documented proxy (hedging language, overconfidence versus support); Layer 2 may add a semantic check. Not true model uncertainty
Internal-state probes	Not implemented. Optional simulated field in the schema, labelled as simulated
Calibrated combiner	A weighted risk score with a threshold sweep. Not calibrated
Pauses the agent before a risky action	Replay of simulated traces with a causal score, so the same logic could run live. We do not claim live prevention
K-Bench and LongFact evaluation, adversarial prompts	Out of scope here; we use our own simulated traces, including a few adversarial-style cases

Fit with the judging criteria

Judges check: step-level localization, interpretable alerts, false-positive awareness, reproducible evaluation. They will ask: what failed, what baseline, what evidence would change your conclusion.

Criterion	How TRACE answers it	Evidence we will show	Fit
Step-level localization	Every step scored; first threshold crossing is the flag	Predicted versus labelled step side by side; exact-hit and within +/-1 rates; blind-spot trace where the baseline misses	Strong by design
Interpretable alerts	Each flag has a signal, a reason, an evidence span and a source badge	Dashboard signal bars per step; intervention rules that cite the signal	Strong by design
False-positive awareness	Clean and tricky-clean traces in the set; a false-alarm rate reported	T01, T02, T08 results; one real false alarm explained; a false-positive note on each intervention rule	Strong if we report real misses
Reproducible evaluation	One command, fixed seed, pinned model, cached LLM outputs	results.json committed; fresh-clone test; held-out results reported separately	Strong if we test it

Answers to the panel's three questions

Panel asks	Our answer
What failed?	Real items from our run: paraphrased claims the rules miss, any LLM false alarms, the T10 case where the baseline does as well, and the simulated probe. We name specific traces.
What baseline?	Final-answer-only check using the same scorers, so the only difference is step-level versus output-only. We also report rules-only and LLM-only rows.
What evidence would change your conclusion?	If the baseline localizes within +/-1 on most traces, step-level adds little here. If catching the blind-spot trace needs a threshold that false-alarms on clean traces, the monitor is unusable. If the hybrid does not beat rules-only on held-out traces, we do not claim the LLM helps. If one signal explains all detections, the others are decoration.

Evaluation plan

Four systems on the same traces, with the same metrics:

System	Layers	Purpose
A. Final-answer-only baseline	Same scorers on the final answer only	Tests the core hypothesis
B. Rules only	Layers 1 + 3	Our deterministic core
C. LLM only	Layers 2 + 3	Shows what the LLM contributes alone
D. Hybrid	Layers 1 + 2 + 3	Our full design; must beat B on held-out traces to count as an improvement

Metrics: exact first-step hit, within +/-1, detection rate, false-alarm rate on clean traces, added latency, per-signal ablation, small threshold sweep. Held-out traces reported separately. **Overlap table:** for every failure trace, record whether it was caught by rules only, LLM only, both, or neither. 'LLM only' is what the LLM actually adds; 'neither' is our honest miss list.

Trace set (13, grow to 16-20 if time allows)

ID	Scenario	Role
T01	Correct workflow, tools agree	Clean reference
T02	Correct, one tool call fails then retries properly	False-positive trap
T03	Incomplete: skips verification, answers anyway	Missing evidence
T04	Two tools return conflicting values	Cross-tool contradiction
T05	Agent states the opposite of the tool result	Consistency failure
T06	Private data leaks in a tool call; final answer looks safe	Blind spot: baseline should miss
T07	Citation to a step that does not support the claim	Evidence failure
T11	Paraphrased contradiction (tool: pending, agent: shipped)	Where rules should fail and the LLM may help
T13	Consistent paraphrase (tool: pending; agent: has not been approved yet)	Hard negative: must NOT be flagged
T08 (held out)	Correct but very hedged language	Uncertainty without error
T09 (held out)	Fails at step 1; agent invents data from an empty result	Earliest possible failure
T10 (held out)	Only the final answer is wrong	Baseline should tie or win; report it
T12 (held out)	Unsupported claim with no numbers to match	Semantic evidence gap

Rules that keep the results honest. (1) Subhiksha writes at least 4 traces before seeing scorer thresholds or LLM prompts. (2) No tuning on held-out traces. (3) If our results look perfect, add a harder trace until something fails. (4) Rule violations cannot be cleared by the LLM; the LLM can only add flags or raise risk. (5) Finalize and commit labels.json before any scorer is run, so no label is adjusted after seeing a result.

Known weaknesses and the safeguard for each

Weakness	Safeguard in this plan
LLM adds nothing where rules already catch the failure	Run rules and LLM independently; report the overlap table; keep the LLM claim only if "LLM only" catches are real on held-out traces
Rules and LLM miss the same failure	Hard cases (T11, T12) plus the T13 hard negative; every "neither" miss analysed and written up; no claim of complete coverage
LLM false alarms and latency	Rules first, LLM only where rules are blind; rule hits never cleared; threshold tuned on dev traces only; latency and cached outputs reported
Risk score is not a calibrated probability	Called a weighted risk score; documented weights and formula; threshold sweep; calibration listed as future work
Few simulated traces	13 to 20 labelled traces with a held-out set; counts, misses and false alarms reported; no real-world effectiveness claim

Task split

Together first (30 min): freeze the trace schema and the signal record; check that a small local model runs on your machines and start the download; send the organizers one line: "Is a locally run open model allowed, given the no-paid-model-calls constraint?" If not confirmed, Layer 2 stays an optional extension.

SARA: monitor, local LLM layer, evaluation

#	Task	Done when
S1	Repo scaffold + schema validator	All traces validate
S2	Evidence scorer (rules): cited steps exist, come earlier, support the claim	Fabricated citation (T07) scores high
S3	Tool-consistency scorer (rules): value mismatches, ignored errors, cross-tool conflicts	T04, T05 flagged at the right step
S4	Uncertainty proxy: hedging, overconfidence versus support; document it as a proxy	T08 visible but not a hard alert
S5	Policy and privacy scorer reading Subhiksha's rules file	T06 flagged at the leaking middle step
S6	Signal record + combiner + first-unreliable-step logic (causal: step t sees steps <= t)	Predicted step matches labels on dev traces
S7	Layer 2: small local model (try an NLI classifier first), structured output with evidence span, temperature 0, cached JSON, on/off switch	T11 behaves differently with Layer 2 on; results reproduce from cache
S8	Final-answer-only baseline + evaluate.py: four systems, metrics, overlap table, ablations, sweep, fixed seed, results.json	One command reproduces every number
S9	Failure analysis: every miss, false alarm, and layer disagreement, with reasons	At least 3 honest failures written down

SUBHIKSHA: data, rules, dashboard, write-up

#	Task	Done when
H1	13+ simulated traces + labels.json (T01, T04, T06 first; at least 4 written blind to scorer code); commit labels before any scorer runs	All validate; held-out set marked; labels locked
H2	SCHEMA.md with field dictionary and a worked example	A stranger could write a trace from it
H3	policy_rules.yaml: forbidden tools, PII patterns, write-without-confirmation, allowlist, unsourced claim rule; plain-English message per rule	Delivered early; Sara builds S5 on it
H4	Offline HTML dashboard: step timeline coloured by risk, predicted vs labelled step side by side, reason chips with rule/LLM badge, four-system comparison panel	Flagged step obvious in 5 seconds; opens without internet
H5	Intervention rules: risk band + dominant signal -> action, each with a false-positive note	Every rule traces to a signal
H6	Flagged examples: a win, the blind-spot catch, a miss, a false alarm, a layer disagreement	Each shows trace, flag, reason
H7	README (reproduce in 3 commands), LIMITATIONS, related-work and scope-versus-abstract sections	Fresh clone runs from the README alone
H8	Review LLM outputs with Sara; write extra paraphrase traces (T11, T12) and the T13 hard negative	Cached outputs sanity-checked
H9	Demo script (90 seconds) and Q&A rehearsal	Both can answer "what failed"

Integration and timeline

Times are relative to **Hour 0** (when you start). Hard rule: the signal record and results.json format are frozen at Hour 0:30. Add fields if needed; never rename or remove them, or the dashboard breaks.

Phase	Sara	Subhiksha	Handoff
0: Contract (0-0:30)	Draft and freeze schema + signal record; check local model runs, start download	Review fields; send the organizer question	**Schema frozen**
1: Core (0:30-6h)	S1-S6 rules pipeline on dev traces	H1 traces (3 early), H2, H3 rules file, dashboard against a hand-made results.json	3 traces at hour 1; rules file by hour 3; all traces by hour 4
Decision point (hour 6)	Rules-only evaluation works end to end?	Dashboard reads real results?	**Yes: start Layer 2. No: finish core first**
2: Hybrid (6-12h)	S7 local LLM layer, cache, switch; S8 baseline + evaluate.py	H8 paraphrase traces and output review; H4 dashboard polish; H5	results.json v1 with four systems
3: Evaluate (12-18h)	S8 final runs on held-out set; S9 failure analysis	H6 flagged examples from real output; H7 write-up	Failures handed over
4: Finish (18-24h+)	Lock results; fresh-clone test	Rebuild report; H9 rehearsal; checklist	**Code freeze**, then submit with buffer

Confirm your real deadline. Earlier you said the prototype is due tomorrow at 10 AM; later you mentioned having a day and a half. This plan fits either: with the longer window you run all phases; if it really is 10 AM tomorrow, do Phases 0-1, rules-only evaluation, dashboard and write-up, and treat Layer 2 as a stretch goal after the decision point.

If you fall behind: cut order (top first)

- 1. Threshold sweep and ablation chart (keep the ablation table)
- 2. Dashboard polish: keep timeline, flagged marker, reason chips, comparison panel
- 3. Layer 2 entirely: ship rules-only and describe the LLM as the next layer
- 4. Traces T02, T03 (keep at least 8, the brief says 5-10 minimum)

Never cut: the baseline, three tests including one edge case, the limitations section, the flagged-step output, the blind-spot trace.

Final checklist

- [ ] Schema + SCHEMA.md
- [ ] 13+ traces + labels, held-out marked, labels committed before scoring
- [ ] Four scorers (evidence, consistency, uncertainty, policy)
- [ ] First unreliable step flagged with reasons on every trace
- [ ] Local LLM layer on/off switch works
- [ ] LLM outputs cached; pinned model recorded
- [ ] Baseline + four-system comparison + overlap table
- [ ] At least 3 tests incl. 1 edge case
- [ ] Dashboard opens offline
- [ ] Flagged examples incl. a miss and a false alarm
- [ ] Intervention rules with false-positive notes
- [ ] LIMITATIONS and scope-versus-abstract written
- [ ] Related work names AgentRx, Who&When
- [ ] Organizer question sent
- [ ] Fresh-clone test passes
- [ ] No credentials, live actions or paid calls